

Removing the common reference requirement from the indirectly invocable concepts

Document #: P2997R0
Date: 2023-10-13
Project: Programming Language C++
Audience: LEWG, SG9
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Tim Song
<t.canens.cpp@gmail.com>

Contents

[1 Introduction](#)

[1.1 Wait... but why?](#)

[1.2 Ok, but... why?](#)

[1.3 Common Reference is for Parameters](#)

[2 Proposal](#)

[2.1 Wording](#)

[3 References](#)

§ 1 Introduction

Consider the following example ([compiler-explorer](#)):

```
#include <algorithm>
#include <ranges>

struct C {
    auto f() -> void;
};

struct Iterator {
    using value_type = C;
    using difference_type = std::ptrdiff_t;
    using iterator_category = std::input_iterator_tag;

    auto operator*() const -> C&&;
    auto operator++() -> Iterator&;
    auto operator++(int) -> void;
    auto operator==(Iterator const&) const -> bool;
};

static_assert(std::input_iterator<Iterator>);
static_assert(std::same_as<std::iter_value_t<Iterator>, C>);
```

```

static_assert(std::same_as<std::iter_reference_t<Iterator>, C&&>);

struct R {
    auto begin() -> Iterator;
    auto end() -> Iterator;
};

static_assert(std::ranges::range<R>);
static_assert(std::same_as<std::ranges::range_reference_t<R>, C&&>);

auto f(R r) -> void {
    std::ranges::for_each(r, [](auto&& c){
        c.f();
    });
}

```

It's a bit lengthy, but the important part is that `R` is a range of `xvalue`¹ `C` - `Iterator` is just a minimal iterator to achieve that goal (the fact that the iterator is just an input iterator doesn't matter, it just makes the example smaller). Note that ranges of `xvalues` already aren't all that obscure and will become even less so with time - `R` has the same properties as `std::generator<C>`.

The above example is trying to simply invoke `c.f()` for every `c` in `r`. When written using a `for` loop, the code compiles. But when written using `std::ranges::for_each`, it does not. The compile error on gcc 13.2 is:

```

/opt/compiler-explorer/gcc-13.2.0/include/c++/13.2.0/type_traits:2558:26:
required by substitution of 'template static std::__result_of_success()
((declval<_Args>())...), std::__invoke_other>
std::__result_of_other_impl::_S_test(int) [with _Fn = f(R)::&; _Args =
{const C&}]'
/opt/compiler-explorer/gcc-13.2.0/include/c++/13.2.0/type_traits:2569:55:
required from 'struct std::__result_of_impl&, const C&>'
/opt/compiler-explorer/gcc-13.2.0/include/c++/13.2.0/type_traits:3077:12:
recursively required by substitution of 'template struct
std::__is_invocable_impl<_Result, _Ret, true, std::__void_t > [with _Result
= std::__invoke_result&, const C&; _Ret = void]'
/opt/compiler-explorer/gcc-13.2.0/include/c++/13.2.0/type_traits:3077:12:
required from 'struct std::is_invocable&, const C&>'
:32:26: recursively required by substitution of 'template requires
(input_range<_Range>) && (indirectly_unary_invocable<_Fun, typename
std::__detail::__projected>())', _Proj>::__type>) constexpr
std::ranges::for_each_result, _Fun> std::ranges::__for_each_fn::operator()
(_Range&&, _Fun, _Proj) const [with _Range = R&; _Proj = std::identity; _Fun
= f(R)::]'
:32:26: required from here
:33:12: error: passing 'const C' as 'this' argument discards qualifiers [-
fpermissive]
    33 |         c.f();
        |         ~~~^~
:5:10: note: in call to 'void C::f()'
     5 |     auto f() -> void;
        |         ^

```

If you stare at the error long enough you might eventually see a hint as to what the problem is: why, exactly, am I getting a compile error about discarding qualifiers on a `const C` when nothing in this code example appears to be forming a `const C`?

§ 1.1 Wait... but why?

The problem here is coming from the algorithm. `ranges::for_each` is declared as:

```
template<input_range R, class Proj = identity,
        indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
constexpr ranges::for_each_result<borrowed_iterator_t<R>, Fun>
    ranges::for_each(R&& r, Fun f, Proj proj = {});
```

To understand what's going on here, we need to look at several other pieces. `projected` is a helper utility that is only used to make it easier to specify constraints (it isn't used in the implementation):

```
template<class I, class Proj>
struct projected_impl {
    struct type {
        using value_type = remove_cvref_t<indirect_result_t<Proj&, I>>;
        using difference_type = iter_difference_t<I>;
        weakly_incrementable
        indirect_result_t<Proj&, I> operator*() const;
    };
};

template<indirectly_readable I, indirectly_regular_unary_invocable<I> Proj>
using projected = projected_impl<I, Proj>::type;
```

In our example, `projected<Iterator, identity>` has the same `value_type` (`C`) and reference (`C&&`) as `Iterator`, so we'll ignore `projected` going forward,² since it doesn't change anything.

The relevant concept for us in this algorithm is this one:

```
template<class F, class I>
concept indirectly_unary_invocable =
    indirectly_readable<I> &&
    copy_constructible<F> &&
    invocable<F&, indirect_value_t<I>> &&
    invocable<F&, iter_reference_t<I>> &&
    invocable<F&, iter_common_reference_t<I>> &&
    common_reference_with<
        invoke_result_t<F&, indirect_value_t<I>>,
        invoke_result_t<F&, iter_reference_t<I>>>;
```

What the code example needs to do is validate that our lambda satisfies

`indirectly_unary_invocable<I>`, where `I` is actually `projected<Iterator, identity>`, but as just noted we can simply treat this as `Iterator`. Before we go through the constraints, it's worth first start with what all the iterator types actually are, because there's a lot and it's easy to get confused.

Associated Type	Type
value_type	C
<i>indirect-value-t</i>	C&
reference	C&&
common_reference	C const &

That the common_reference between C& and C&& comes out as C **const**& might be initially surprising, but really the only other type it could be is C - and that would mean that initializing a common *reference* would always be either a copy (from C&) or move (from C&&), which isn't really what we're going for here.

As a result, if we go through our four constraints:

Constraint	Outcome
invocable<F&, <i>indirect-value-t</i> <I>>	✓
invocable<F&, iter_reference_t<I>>	✓
invocable<F&, iter_common_reference_t<I>>	✗
common_reference_with<..., ...>	✓

That third one fails, because we're trying to invoke our lambda with a C **const**&, which instantiates the lambda - which doesn't have any constraints - and the instantiation of the body (to figure out the return type) causes the compiler to instantiate that `c.f()` call, which isn't valid for a C **const**. Hard error.

§ 1.2 Ok, but... why?

We have to go back to Eric Niebler's four-part series on iterators [[niebler.iter.0](#)] [[niebler.iter.1](#)] [[niebler.iter.2](#)] [[niebler.iter.3](#)]. There, he introduced an algorithm `unique_copy` which looks like this:

```

1 // Copyright (c) 1994
2 // Hewlett-Packard Company
3 // Copyright (c) 1996
4 // Silicon Graphics Computer Systems, Inc.
5 template <class InIter, class OutIter, class Fn,
6          class _Tp>
7 OutIter
8 __unique_copy(InIter first, InIter last,
9              OutIter result,
10              Fn binary_pred, _Tp*) {
11     _Tp value = *first;
12     *result = value;
13     while (++first != last)
14         if (!binary_pred(value, *first)) {
15             value = *first;
16             *++result = value;
17         }
18     return ++result;
19 }
```

[...] Note the value local variable on line 11, and especially note line 14, where it passes a value and a reference to `binary_pred`. Keep that in mind because it's important!

[...] Why do I bring it up? Because it's *super problematic* when used with proxy iterators. Think about what happens when you try to pass `vector<bool>::iterator` to the above `__unique_copy` function:

```
std::vector<bool> vb{true, true, false, false};
using R = std::vector<bool>::reference;
__unique_copy(
    vb.begin(), vb.end(),
    std::ostream_iterator<bool>{std::cout, " "},
    [](R b1, R b2) { return b1 == b2; }, (bool*)0 );
```

This *should* write a “true” and a “false” to `cout`, but it doesn't compile. Why? The lambda is expecting to be passed two objects of `vector<bool>`'s proxy reference type, but remember how `__unique_copy` calls the predicate:

```
if (!binary_pred(value, *first)) { /*...*/
```

That's a `bool&` and a `vector<bool>::reference`. Ouch!

The blog goes on to point out that one way to resolve this is by having the lambda take both parameters as `auto&&`. But having to *require* a generic lambda is a bit... much. This was the reason we have the idea of a `common_reference`. Rewriting the above to be:

```
using R = std::iter_common_reference_t<std::vector<bool>::iterator>;
__unique_copy(
    vb.begin(), vb.end(),
    std::ostream_iterator<bool>{std::cout, " "},
    [](R b1, R b2) { return b1 == b2; }, (bool*)0 );
```

This now works. And that is now the requirement that we have for iterators, that they be `indirectly_readable` - which, among other things, requires that there be a `common_reference` between the iterator's reference type and the iterator's `value_type&`.

That is: the common reference is the type that you can use as the parameter of a non-generic callable that you pass into an algorithm, to ensure that both forms (value and `*it` above) that the implementation might use to invoke the callable work.

§ 1.3 Common Reference is for Parameters

The key thing to point out in the above motivation though is that the common reference type is useful *for parameters*. That is, for the user invoking the algorithm to be able to do so with a homogenous callable - a non-generic lambda.

But what the common reference type is *not* useful for is the algorithm implementations themselves. The `unique_copy` implementation above is never going to construct a `std::iter_common_reference<InIter>`. It has no reason to do so - it's only going to traffic in

`std::iter_value_t<InIter>&` and `std::iter_reference_t<InIter>`. The same is true for every algorithm.

The only places in the standard library right now where we actually do construct common references are either:

- when we actually do need the common reference of multiple ranges (e.g. `views::join_with` and the proposed `views::concat`), since we’re explicitly doing merging
- when we’re trying to produce a new reference type (e.g. `views::as_const`)

But internally within the algorithms? No such need.

So let’s go back to the definition of `std::indirectly_unary_invocable`, which was the constraint the original example failed:

```
1  template<class F, class I>
2      concept indirectly_unary_invocable =
3          indirectly_readable<I> &&
4          copy_constructible<F> &&
5          invocable<F&, indirect_value_t<I>> &&
6          invocable<F&, iter_reference_t<I>> &&
7          invocable<F&, iter_common_reference_t<I>> &&
8          common_reference_with<
9              invoke_result_t<F&, indirect_value_t<I>>,
10             invoke_result_t<F&, iter_reference_t<I>>>;
```

Excluding the check that `I` is an iterator and `F` is copyable, we have 4 other checks here:

1. that `F&` is invocable with `iter_value_t<I>&` (`indirect_value_t<I>` is basically this, but in a way that will work correctly through projections, which we don’t have to worry about in this paper)
2. that `F&` is invocable with `iter_reference_t<I>`
3. that `F&` is invocable with `iter_common_reference_t<I>`
4. that the results of invocations (1) and (2) have a common reference

Notably, (3) is kind of the odd man out in this concept. Algorithms certainly may need to invoke `F` with `iter_value_t<I>&` and with `iter_reference_t<I>` (as in the `unique_copy` example above, which does both). But algorithms don’t need to internally construct a common reference ever.

Moreover, we check we can invoke `F` with the common reference - but while we check that value/reference invocations are compatible, we don’t actually check that the common reference invocation is compatible with... either of the other two? It could just do something different entirely? It’s not clear what the point of this part of the concept actually is.

§ 2 Proposal

The common reference type exists to be able to merge multiple ranges and to provide users with a way to write a non-generic callable. But the requirement that callables be invocable with the iterator’s common

reference type isn't a useful requirement for any algorithm. And while we require this invocability, we don't even require that this invocation (which no algorithm uses) is compatible with the invocations that the algorithms do use. This check seems to simply reject valid code (as in the original example) while providing no value.

We propose to simply remove this check from every indirect invocation concept. That would fix the above example and others like it, while also not removing a requirement that algorithms actually rely on. Those uses cited above that actually do need to produce a common reference (`views::join_with`, `views::concat`, and `views::as_const`) already have this requirement separately.

§ 2.1 Wording

Remove all the `iter_common_reference_t` invocation requirements in 25.3.6.3

[\[indirectcallable.indirectinvocable\]](#) (which frequently requires a multi-line diff just because the common reference invocation was the last one in the list):

```
namespace std {
    template<class F, class I>
    concept indirectly_unary_invocable =
        indirectly_readable<I> &&
        copy_constructible<F> &&
        invocable<F&, indirect_value_t<I>> &&
        invocable<F&, iter_reference_t<I>> &&
-       invocable<F&, iter_common_reference_t<I>> &&
        common_reference_with<
            invoke_result_t<F&, indirect_value_t<I>>,
            invoke_result_t<F&, iter_reference_t<I>>>;

    template<class F, class I>
    concept indirectly_regular_unary_invocable =
        indirectly_readable<I> &&
        copy_constructible<F> &&
        regular_invocable<F&, indirect_value_t<I>> &&
        regular_invocable<F&, iter_reference_t<I>> &&
-       regular_invocable<F&, iter_common_reference_t<I>> &&
        common_reference_with<
            invoke_result_t<F&, indirect_value_t<I>>,
            invoke_result_t<F&, iter_reference_t<I>>>;

    template<class F, class I>
    concept indirect_unary_predicate =
        indirectly_readable<I> &&
        copy_constructible<F> &&
        predicate<F&, indirect_value_t<I>> &&
-       predicate<F&, iter_reference_t<I>> &&
-       predicate<F&, iter_common_reference_t<I>>;
+       predicate<F&, iter_reference_t<I>>;

    template<class F, class I1, class I2>
    concept indirect_binary_predicate =
        indirectly_readable<I1> && indirectly_readable<I2> &&
        copy_constructible<F> &&
```

```

    predicate<F&, indirect-value-t<I1>, indirect-value-t<I2>> &&
    predicate<F&, indirect-value-t<I1>, iter_reference_t<I2>> &&
    predicate<F&, iter_reference_t<I1>, indirect-value-t<I2>> &&
-   predicate<F&, iter_reference_t<I1>, iter_reference_t<I2>> &&
-   predicate<F&, iter_common_reference_t<I1>,
    iter_common_reference_t<I2>>;
+   predicate<F&, iter_reference_t<I1>, iter_reference_t<I2>>;

template<class F, class I1, class I2 = I1>
concept indirect_equivalence_relation =
    indirectly_readable<I1> && indirectly_readable<I2> &&
    copy_constructible<F> &&
    equivalence_relation<F&, indirect-value-t<I1>, indirect-value-t<I2>>
    &&
    equivalence_relation<F&, indirect-value-t<I1>, iter_reference_t<I2>>
    &&
    equivalence_relation<F&, iter_reference_t<I1>, indirect-value-t<I2>>
    &&
-   equivalence_relation<F&, iter_reference_t<I1>, iter_reference_t<I2>>
    &&
-   equivalence_relation<F&, iter_common_reference_t<I1>,
    iter_common_reference_t<I2>>;
+   equivalence_relation<F&, iter_reference_t<I1>, iter_reference_t<I2>>;

template<class F, class I1, class I2 = I1>
concept indirect_strict_weak_order =
    indirectly_readable<I1> && indirectly_readable<I2> &&
    copy_constructible<F> &&
    strict_weak_order<F&, indirect-value-t<I1>, indirect-value-t<I2>> &&
    strict_weak_order<F&, indirect-value-t<I1>, iter_reference_t<I2>> &&
    strict_weak_order<F&, iter_reference_t<I1>, indirect-value-t<I2>> &&
-   strict_weak_order<F&, iter_reference_t<I1>, iter_reference_t<I2>> &&
-   strict_weak_order<F&, iter_common_reference_t<I1>,
    iter_common_reference_t<I2>>;
+   strict_weak_order<F&, iter_reference_t<I1>, iter_reference_t<I2>>;
}

```

§ 3 References

[LWG3859] Hewill Kang. std::projected cannot handle proxy iterator.

<https://wg21.link/lwg3859>

[niebler.iter.0] Eric Niebler. 2015. To Be or Not to Be (an Iterator).

<http://ericniebler.com/2015/01/28/to-be-or-not-to-be-an-iterator/>

[niebler.iter.1] Eric Niebler. 2015. Iterators++, Part 1.

<http://ericniebler.com/2015/02/03/iterators-plus-plus-part-1/>

[niebler.iter.2] Eric Niebler. 2015. Iterators++, Part 2.

<http://ericniebler.com/2015/02/13/iterators-plus-plus-part-2/>

[niebler.iter.3] Eric Niebler. 2015. Iterators++, Part 3.

<http://ericniebler.com/2015/03/03/iterators-plus-plus-part-3/>

-
1. The exact same issue comes up for a range of prvalues, arguably in a more surprising way. But a range of prvalue is effectively turned into a range of xvalue by way of projecting identity, so it's the same behavior. Note also that the use identity doesn't prevent prvalue elision - `std::invoke` already does that.[↵](#)
 2. [\[LWG3859\]](#) had suggested to special casing projected for identity to solve a related issue, and as the previous footnote indicates, had `Iterator`'s reference been a prvalue, it would look like the use identity as the projection actually caused the issue. But that would be a red herring, which is why we're using an `Iterator` over xvalues as the example instead.[↵](#)